



Path Planning Task

Path Generation

Prepared by:

Amir Atef Abdullah

Author's Note

- All the diagrams in this report were created by me, and the original `draw.io` file is pushed onto the repo

[i rendered a manim animation to illustrate this task](#)

1 Introduction and Problem Scope

In Formula Student Artificial Intelligence (FSAI) competitions, an autonomous racecar navigates through an unknown track marked by colored cones. The car lacks a global map. It only sees cones within its sensor range and must compute a path between the boundaries.

Assessment Scope

Assessment Requirements:

- **Track Color Rules:** Yellow cones (`color = 0`) define the right side of the track. Blue cones (`color = 1`) define the left side.
- **Planner Inputs:** Current car pose (x, y, yaw) in meters and radians, plus a list of detected cones.
- **Part 1 Scope:** Handle up to two cones per boundary (0, 1, or 2 on either side).
- **Part 2 Scope:** Extend the planner to handle three cones on one side of the track, add custom test scenarios, and analyze limitations.
- **Path Constraints:** Total path length must be between 5 and 10 meters, with point spacing ≤ 0.5 meters.

Sensors miss cones due to distance or occlusions. The planner might receive paired cones, cones on one side, or no cones at all. Figure 1 shows the main cone combinations from Part 1

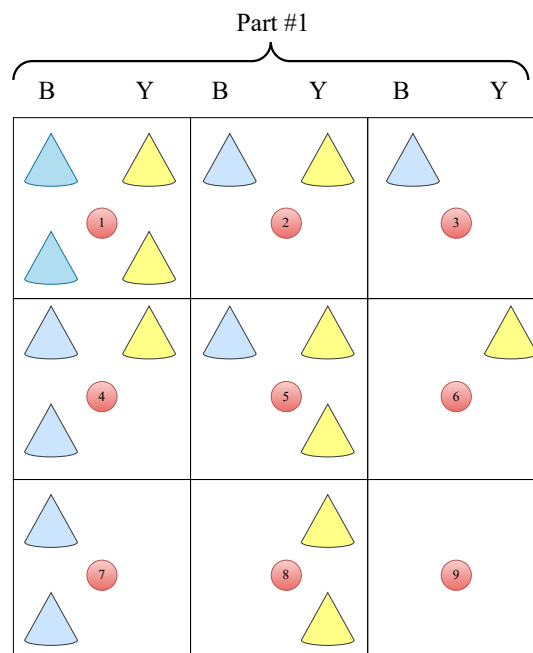


Figure 1: Part 1 cone combinations

2 System Overview and Architecture

The project uses three data classes in `src/models.py`:

- **Cone**: Stores world coordinates (x, y) and an integer `color` (0 for yellow, 1 for blue).
- **CarPose**: Stores the car position (x, y) and heading angle `yaw` (radians from $+X$).
- **Path2D**: A type alias for a list of (x, y) tuples representing the planned path.

The planning logic is done by `generatePath()` method of `PathPlanning` in `src/path_planning.py`. Figure 2 maps the pipeline.

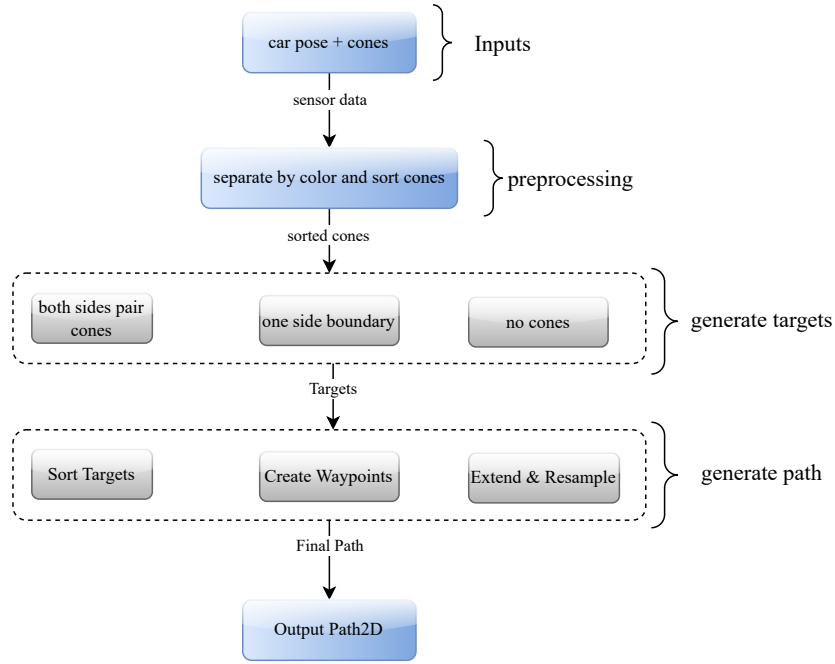


Figure 2: Architecture pipeline

The pipeline reads the car pose and cones, separates cones by color, and sorts them by distance from car. It builds target waypoints based on cone availability, extends the path to 7.0 meters, and resamples points every 0.25 meters.

3 Geometric Path Planning Logic

Figure 3 shows the logic inside `generatePath()`. The planner determines which boundaries are visible and executes one of four geometric cases.

3.1 Preprocessing

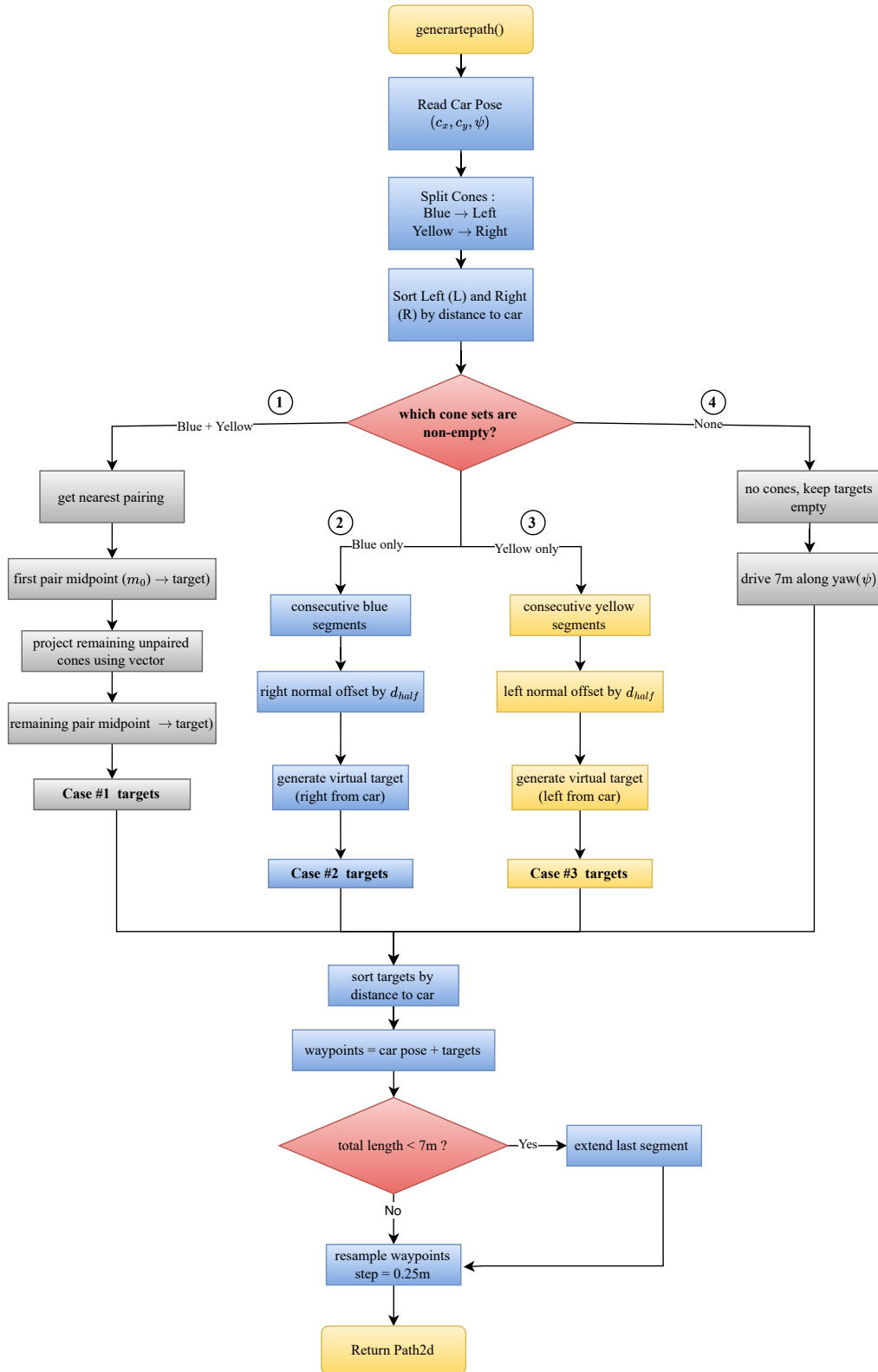
The algorithm splits cones into two lists: `blue` (left) and `yellow` (right). Both lists are sorted by Euclidean distance to the car:

$$d = \sqrt{(\text{cone.x} - \text{cx})^2 + (\text{cone.y} - \text{cy})^2} \quad (1)$$

so the planner targets the closest cones first.

3.2 Case 1: Cones on Both Sides

When cones appear on both sides, the track has two boundaries. One side often has more cones than the other. The planner compares the list lengths, calls the shorter list `short_list` and the longer one `long_list`.

Figure 3: Flowchart of `generatePath()`

For each cone in `short_list`, the algorithm finds the closest unused cone in `long_list`. The midpoint of this first pair becomes the first target:

$$x_m = \frac{b0.x + y0.x}{2}, \quad y_m = \frac{b0.y + y0.y}{2} \quad (2)$$

Figure 4 illustrates this calculation.

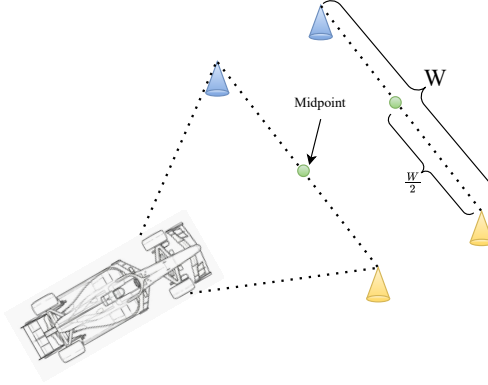


Figure 4: Midpoint target between paired yellow and blue cones.

Extra unpaired cones in `long_list` indicate where the track curves. The planner uses the first vector $v_{yb} = (b0.x - y0.x, b0.y - y0.y)$ to estimate track width and direction. It shifts yellow unpaired cones by $+0.5 v_{yb}$ and blue ones by $-0.5 v_{yb}$ toward the center. Remaining paired cones add their midpoints.

3.3 Case 2: Only Blue Cones Visible

When only blue cones appear, the right boundary is missing. The algorithm offsets the path to the right by `d_half` = 1.0 meter.

Figure 5 shows the geometry. For two consecutive blue cones, the tangent vector is:

$$tx = blue[i+1].x - blue[i].x, \quad ty = blue[i+1].y - blue[i].y, \quad l = \sqrt{tx^2 + ty^2} \quad (3)$$

The algorithm rotates the tangent vector 90 degrees clockwise to point inward (right):

$$nx = \frac{ty}{l}, \quad ny = -\frac{tx}{l} \quad (4)$$

Waypoints are placed at $(cone.x + d_half \times nx, cone.y + d_half \times ny)$.

For a single blue cone, the planner rotates the car-to-cone vector 90 degrees clockwise to get

$$(uy/l, -ux/l)$$

.

3.4 Case 3: Only Yellow Cones Visible

When only yellow cones appear, the vehicle offsets left by 1.0 meter.

As shown in Figure 6, for consecutive yellow cones, the algorithm rotates the tangent vector 90 degrees counter-clockwise:

$$nx = -\frac{ty}{l}, \quad ny = \frac{tx}{l} \quad (5)$$

Waypoints are placed at $(cone.x + d_half \times nx, cone.y + d_half \times ny)$. For a single yellow cone, the car-to-cone vector rotated counter-clockwise provides the normal.

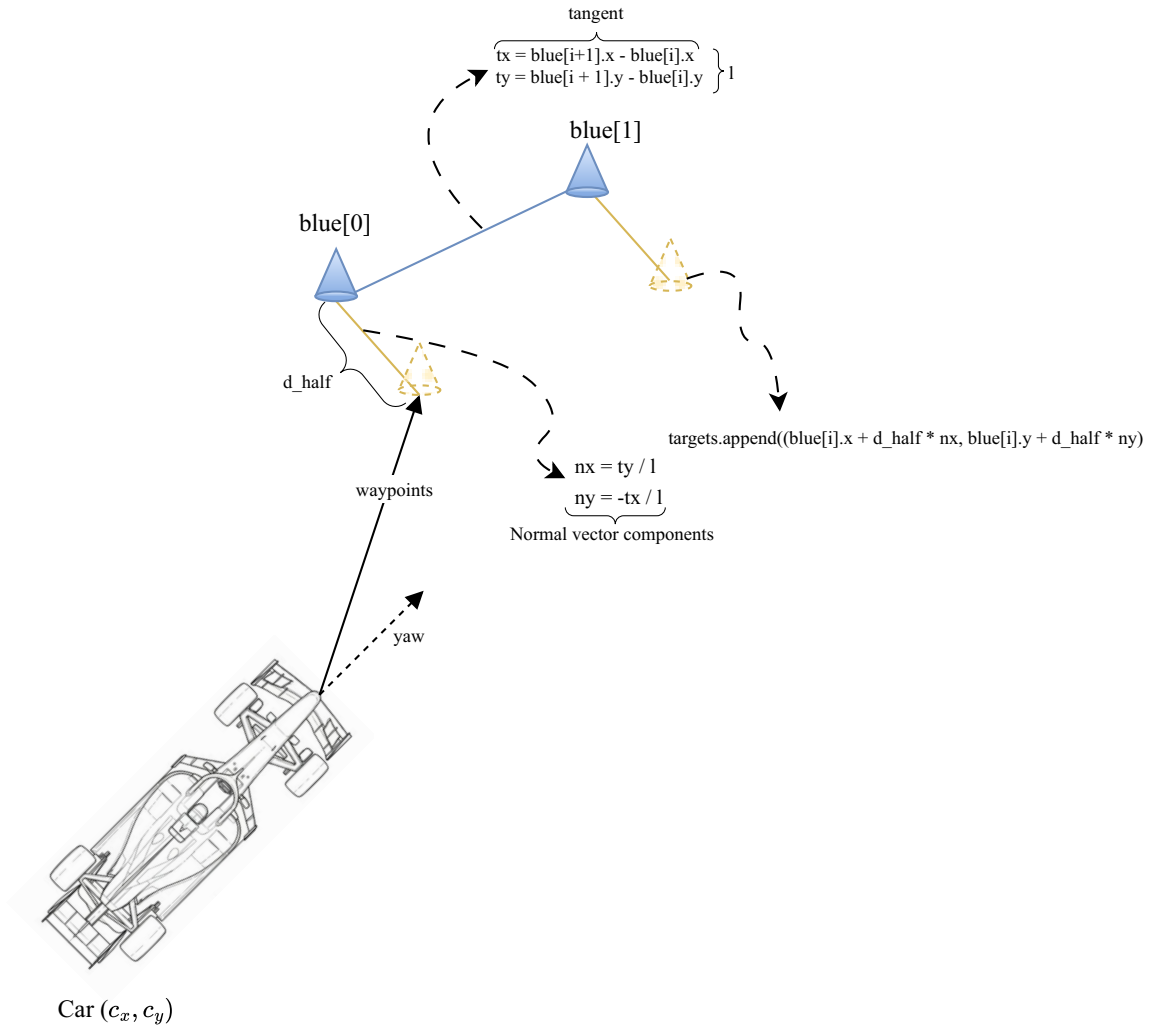


Figure 5: Case 2: Left boundary tracking (Blue cones).

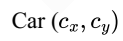


Figure 6: Case 3: Right boundary tracking (Yellow cones).

3.5 Case 4: No Cones Visible

If the sensors detect no cones, the planner relies on [dead reckoning](#), It projects a straight line 7.0 meters ahead along the car heading:

$$cx + \text{target_len} \times \cos(\text{yaw}), \quad cy + \text{target_len} \times \sin(\text{yaw}) \quad (6)$$

3.6 Waypoint Extension and Uniform Resampling

After generating target points, the planner sorts them by distance from the car and inserts the car pose at the beginning: `waypoints = [(cx, cy)] + targets`.

The algorithm sums the segment lengths. If the total distance is less than 7.0 meters, it extends along the direction of the final segment until the length reaches 7.0 meters.

To finish the path, the planner resamples the piecewise geometry with a constant step size `step = 0.25` meters. Using cumulative arc length spaces points at uniform intervals along the line.

4 Implementation

the following are three code sections from `src/path_planning.py`.

4.1 Pairing and Unpaired Projection (Case 1)

Listing 1 shows cone pairing and unpaired cone projection.

```

1      used_indices: List[int] = []
2      pairs = []
3      for sc in short_list:
4          best_idx = -1
5          best_dist = 9999999
6          for i in range(len(long_list)):
7              if i not in used_indices:
8                  d = distance(sc.x, sc.y, long_list[i].x, long_list[i].y)
9                  if d < best_dist:
10                     best_dist = d
11                     best_idx = i
12             used_indices.append(best_idx)
13             pairs.append((sc, long_list[best_idx]))
14
15         # Midpoint of first pair
16         sc0, lc0 = pairs[0]
17         if is_short_blue:
18             b0, y0 = sc0, lc0
19         else:
20             b0, y0 = lc0, sc0
21         targets.append(((b0.x + y0.x) / 2.0, (b0.y + y0.y) / 2.0))
22
23         # Project remaining unpaired cone
24         v_yb = (b0.x - y0.x, b0.y - y0.y)
25         for i in range(len(long_list)):
26             if i not in used_indices:
27                 lc = long_list[i]
28                 if is_short_blue:
29                     targets.append((lc.x + 0.5 * v_yb[0], lc.y + 0.5 * v_yb[1]))
30                 else:
31                     targets.append((lc.x - 0.5 * v_yb[0], lc.y - 0.5 * v_yb[1]))

```

Listing 1: cones on both sides + unpaired cone projection

4.2 Single-Boundary Tangent and Normal Calculation (Case 2 & 3)

Listing 2 shows the tangent and normal rotation logic for blue and yellow boundaries.

```

1      # Case 2: Only blue cones
2      elif len(blue) > 0:
3          if len(blue) >= 2:
4              for i in range(len(blue) - 1):
5                  tx = blue[i + 1].x - blue[i].x
6                  ty = blue[i + 1].y - blue[i].y
7                  l = distance(blue[i].x, blue[i].y, blue[i + 1].x, blue[i + 1].y)
8                  if l > 0.00001:
9                      nx = ty / l
10                     ny = -tx / l
11                     if i == 0:
12                         targets.append((blue[0].x + d_half * nx, blue[0].y + d_half * ny))
13                     targets.append((blue[i + 1].x + d_half * nx, blue[i + 1].y + d_half * ny))
14                     if len(targets) == 0:
15                         ux = blue[0].x - cx
16                         uy = blue[0].y - cy
17                         l = distance(cx, cy, blue[0].x, blue[0].y)
18                         if l > 0.00001:
19                             nx = uy / l
20                             ny = -ux / l
21                         targets.append((blue[0].x + d_half * nx, blue[0].y + d_half * ny))
22                     else:
23                         targets.append((blue[0].x + d_half * math.sin(yaw), blue[0].y - d_half * math.cos(yaw)))

```

Listing 2: Inward normal calculation for single-sided boundaries.

The 2D rotation formulas keep the normal vector pointing into the track corridor.
The check 0.00001 prevents division by zero.

4.3 Path Extension and Resampling

Listing 3 handles horizon extension and arc-length resampling.

Advancing `seg_idx` through the cumulative distances keeps the resampling routine at $O(N)$ runtime.

```

1      # Extend waypoints to reach target length (7m)
2      target_len = 7.0
3      cumulative_len = 0.0
4      for i in range(len(waypoints) - 1):
5          cumulative_len += distance(waypoints[i][0], waypoints[i][1], waypoints[i + 1][0], waypoints[i +
        ↪ 1][1])
6
7      if len(waypoints) == 1:
8          # Case #4 / No cones (drive along yaw)
9          waypoints.append((cx + target_len * math.cos(yaw), cy + target_len * math.sin(yaw)))
10         elif cumulative_len < target_len:
11             dx = waypoints[-1][0] - waypoints[-2][0]
12             dy = waypoints[-1][1] - waypoints[-2][1]
13             l = distance(waypoints[-2][0], waypoints[-2][1], waypoints[-1][0], waypoints[-1][1])
14             rem = target_len - cumulative_len
15             if l > 0.00001:
16                 waypoints.append((waypoints[-1][0] + (dx / l) * rem, waypoints[-1][1] + (dy / l) * rem))
17             else:
18                 waypoints.append((waypoints[-1][0] + rem * math.cos(yaw), waypoints[-1][1] + rem * math.sin(yaw)))
19
20         # Resample waypoints
21         step = 0.25
22         dists = [0.0]
23         for i in range(len(waypoints) - 1):
24             d = distance(waypoints[i][0], waypoints[i][1], waypoints[i + 1][0], waypoints[i + 1][1])
25             dists.append(dists[-1] + d)
26
27         total_dist = dists[-1]
28         path: Path2D = [(cx, cy)]
29         s = step
30         seg_idx = 0
31         while s <= total_dist:
32             while seg_idx < len(dists) - 1 and dists[seg_idx + 1] < s:
33                 seg_idx += 1
34             seg_start = dists[seg_idx]
35             seg_len = dists[seg_idx + 1] - dists[seg_idx]
36             t = (s - seg_start) / seg_len if seg_len > 0.00001 else 0.0
37             x = waypoints[seg_idx][0] + t * (waypoints[seg_idx + 1][0] - waypoints[seg_idx][0])
38             y = waypoints[seg_idx][1] + t * (waypoints[seg_idx + 1][1] - waypoints[seg_idx][1])
39             path.append((x, y))
40             s += step

```

Listing 3: Horizon extension to 7.0 meters and 0.25-meter uniform resampling.

5 Part 2: Three Cones on One Side

Part 2 tests how the algorithm handles three cones on one boundary.

Assessment Theory Question

Part 2 Question: *What if you were given three cones on one side of the track? Implement your approach to use the new givens. Add new test cases to test your approach in `src/scenarios.py`. Explain why you chose this solution and what could be its limitations.*

5.1 Handling Three Cones

The algorithm handles three or more cones without special branch checks

$$tx = \text{cone}[i+1].x - \text{cone}[i].x, \quad ty = \text{cone}[i+1].y - \text{cone}[i].y \quad (7)$$

runs across consecutive cone pairs. With three cones on one boundary, the loop creates two segments and places inward offset waypoints at all three cone positions. This creates a multi-segment curve bending along the boundary.

With three yellow cones on one side and one blue cone on the other, the planner matches the first gate and projects the remaining two yellow cones across using vector $\mathbf{v}_{yb}$.

5.2 Custom Test Scenarios Added

Three custom scenarios in `src/scenarios.py` test Part 2:

- **Scenario 21:** Three blue cones at (1.0, 2.0), (2.5, 3.5), (4.0, 4.5). Tests two consecutive left segments along a leftward curve.
- **Scenario 22:** Three yellow cones at (2.0, 1.0), (3.5, 1.5), (5.0, 3.0). Tests two consecutive right segments along an expanding right turn.
- **Scenario 23:** One blue cone at (1.0, 3.0) and three yellow cones at (1.0, 1.0), (3.0, 1.0), (5.0, 2.0). Tests single-gate matching combined with multi-cone projection.

5.3 Limitations of this Approach

This approach has two limitations in tight turns:

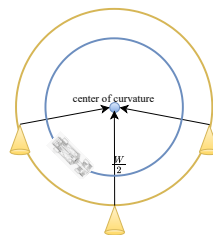


Figure 7: Local radius of curvature ρ in sharp corners.

One drawback of calculating each segment's normal independently is that on tight inside corners with a small radius of curvature (Figure 7), the inward normal vectors end up crossing each other, which creates sharp discontinuities in the generated waypoints.

Another limitation is that uneven cone spacing would skew the segment tangents toward the wider gaps. however the current method works perfectly well for a short 7-meter horizon(although cubic splines would improve the steering angles for driving at higher speeds)

6 Scenario Testing and Results

The planner was tested across 23 scenarios. Table 1 covers the four cases and the Part 2 multi-cone tests

Table 1: Results across representative test scenarios.

Run	Cones (B, Y)	Yaw [rad]	Length [m]	Points	Observed Path Behavior
01	(0, 0)	0.00	7.00	29	Drives straight along heading $\theta = 0$.
03	(2, 2)	0.80	6.99	29	Straight centerline through two symmetric cones
05	(0, 2)	1.60	7.00	29	Follows yellow boundary with 1.0 m leftward offset.
07	(2, 1)	1.40	6.98	29	Pairs first cone, projects extra blue cone along cone vector.
08	(2, 0)	1.80	6.99	29	Follows blue boundary with 1.0 m rightward offset.
09	(1, 0)	0.20	6.75	28	Single blue cone, derives normal from car-to-cone line.
11	(2, 2)	1.00	6.98	29	Curved dual boundary; smooth path through offset cones.
14	(1, 2)	5.20	6.98	29	Large initial yaw; steers car into corridor without sharp corrections.
21	(3, 0)	0.80	7.00	29	Part 2: Chains two blue segments along left curve.
22	(0, 3)	0.30	6.98	29	Part 2: Chains two yellow segments along right curve.
23	(1, 3)	0.40	6.99	29	Part 2: Pairs first cone and projects two yellow cones.

6.1 Discussion of Test Runs

Figure 8 compares two-boundary scenarios. In Scenario 03, the planner matches both cones and places midpoints at (1.0, 2.0) and (3.0, 2.0). In Scenario 11. In Scenario 07, the planner projects the second blue cone downward using the first gate's orientation, keeping the car centered.

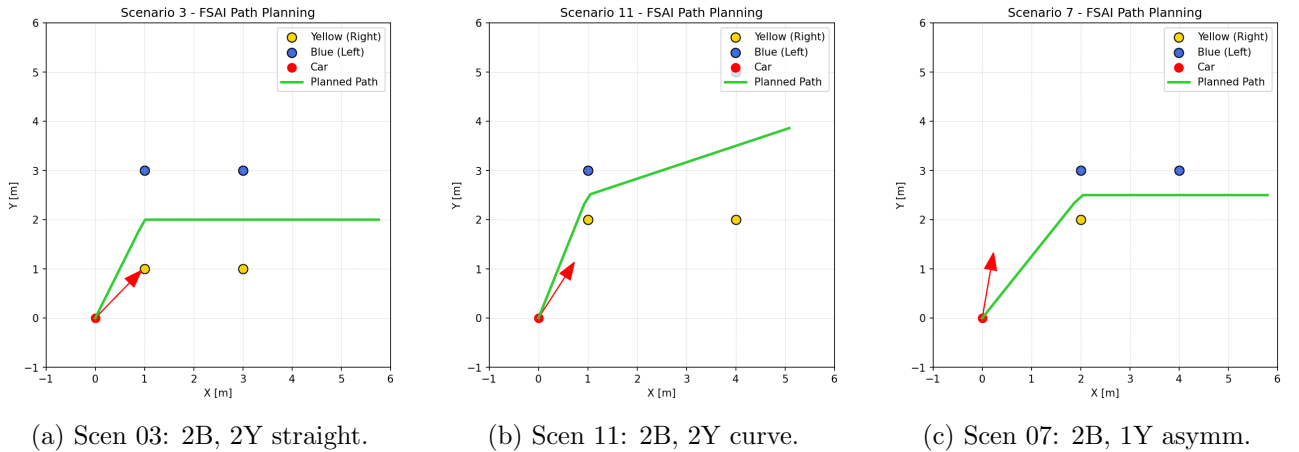


Figure 8: Dual-boundary, curved, and asymmetric test results.

Figure 9 shows single-boundary and Part 2 multi-cone tests. Scenario 08 demonstrates wall-following behavior, maintaining the 1.0-meter offset from the blue boundary. Scenario 21 confirms that three cones produce a smooth two-segment path around corners. Scenario 23 uses one gate pair and two projected cones to steer the car through an asymmetric turn.

The output path stays between 6.75 and 7.00 meters long in all scenarios. Points are spaced every 0.25 meters.

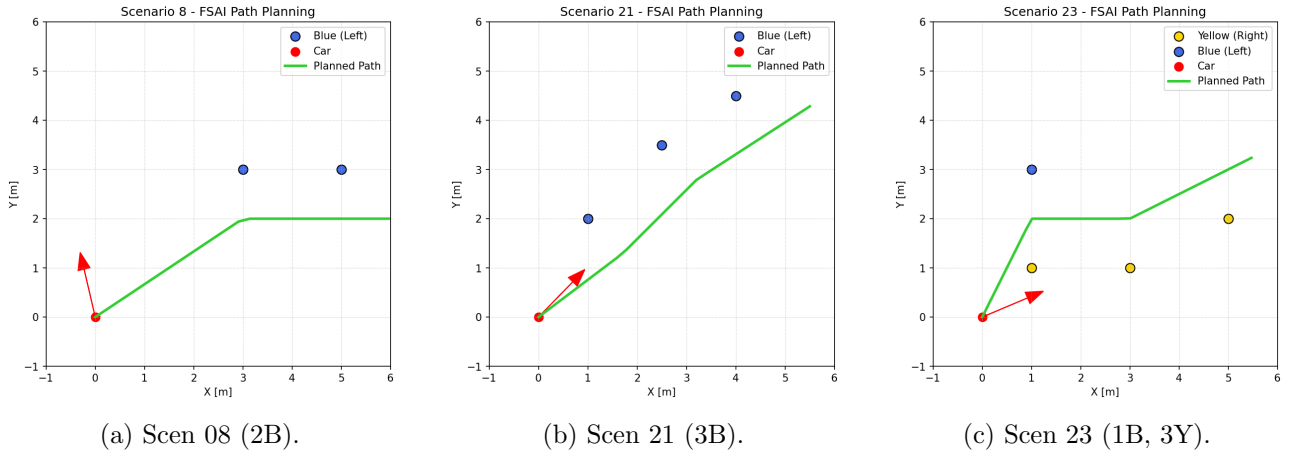


Figure 9: Single-boundary and Part 2 multi-cone test results.

7 Discussion and Trade-Offs

7.1 Successes

- **Speed:** The implementation executes in less than 0.5 milliseconds per frame using 2D arithmetic.
- **Robustness:** The planner processes 0, 1, 2, or 4 cones without throwing exceptions.
- **Clean Resampling:** Uniform 0.25-meter steps give the vehicle controller predictable point spacing.

7.2 Practical Limitations

- **Fixed Clearance:** A fixed 1.0-meter offset works well on the 2-meter test grid. Formula Student tracks span 3 to 4 meters. On a wider track, the car hugs the visible boundary instead of centering itself.
- **No Kinematic Limits:** The planner outputs geometric lines without checking if the steering actuator can follow them.
- **Straight Extrapolation:** Extrapolating from the last segment assumes the track continues straight past the sensor range.

8 Conclusion

The geometric path planner navigates cone-marked tracks using four boundary cases. The planner pairs cones, projects unpaired cones, offsets single boundaries by 1.0 meter, and extrapolates paths to 7.0 meters with 0.25-meter point spacing. The loop-based design handles Part 2 multi-cone setups. Test runs across 23 scenarios confirmed the planner generates valid, centered trajectories.

A Variable Abbreviations Glossary

Table 2: Glossary of code abbreviations

Variable	Type / Unit	Meaning
<code>cx, cy</code>	float [m]	World coordinates of the car position.
<code>yaw</code>	float [rad]	Vehicle heading angle relative to $+X$.
<code>d_half</code>	float [m]	Assumed half-track clearance offset (1.0 m)
<code>sc, lc</code>	Cone	Cones from the shorter and longer boundary lists
<code>v_yb</code>	tuple [m]	Track vector pointing from yellow cone to blue cone
<code>tx, ty</code>	float [m]	Tangent vector connecting consecutive cones
<code>l</code>	float [m]	Euclidean length of a tangent or distance vector
<code>nx, ny</code>	float	Inward unit normal vector components
<code>ux, uy</code>	float [m]	Displacement vector from car to an isolated cone
<code>target_len</code>	float [m]	Target path horizon length (7.0 m)
<code>cumulative_len</code>	float [m]	Cumulative length of the raw waypoint chain
<code>rem</code>	float [m]	Remaining distance needed to reach 7.0 meters
<code>s, step</code>	float [m]	Arc length parameter and discrete step size (0.25 m)
<code>seg_idx</code>	int	Index of the active waypoint segment during resampling